OOP FOR AI APPLICATIONS

Modular Code Structure • Error Handling • Custom Exceptions

A comprehensive technical presentation for ML engineers and software architects

Course Agenda

This document covers four interconnected pillars of professional AI engineering:

#	Topic	Key Focus
1	OOP Foundations for AI	Classes, Objects, Encapsulation
2	Core OOP Principles	SOLID, Inheritance, Polymorphism
3	Modular Code Architecture	Packages, Interfaces, Dependency Injection
4	Design Patterns in AI	Factory, Strategy, Observer, Pipeline
5	Error Handling Strategies	Try/Except, Logging, Graceful Degradation
6	Custom Exception Hierarchies	Domain Errors, Typed Exceptions, Chaining
7	AI-Specific OOP Patterns	Model Wrappers, Data Pipelines, Agents
8	Testing OOP AI Code	Unit Tests, Mocks, Fixtures

© AI Engineering — Confidential

Why OOP Matters in AI Engineering

AI systems are not just algorithms — they are complex, evolving software products. Object-Oriented Programming (OOP) gives engineers the structural tools to manage that complexity sustainably.

The Problem with Procedural AI Code

- Scripts grow into monoliths — a single 2,000-line training file becomes unmaintainable
- No clear ownership of data transformations, model logic, or evaluation steps
- Changing one hyperparameter can silently break an unrelated preprocessing step
- Reproducing experiments becomes difficult without isolated, versioned components

What OOP Solves

Challenge	OOP Solution
Tangled preprocessing + model logic	Encapsulate each concern in its own class
Hard-coded model types	Abstract base class + concrete implementations
Silent failures in inference pipelines	Typed custom exceptions with context
Difficult unit testing	Dependency injection & mockable interfaces
Code reuse across projects	Modular packages with clear contracts

Key Insight

In production AI, the model is 5% of the codebase. The other 95% — data pipelines, serving infrastructure, monitoring, and orchestration — benefits enormously from solid OOP design.

OOP Foundations for AI Systems

Classes & Objects — The Building Blocks

A class is a blueprint; an object is a live instance. In AI, nearly every concept — a dataset, a model, a tokenizer, an evaluation metric — can be represented as a class.

- Class
- Object
- Constructor (__init__)
- Self

Example: A simple ModelConfig class

```
class ModelConfig:
    def __init__(self, model_name: str, max_tokens: int, temperature: float):
        self.model_name   = model_name
        self.max_tokens   = max_tokens
        self.temperature  = temperature

    def is_valid(self) -> bool:
        return 0.0 <= self.temperature <= 2.0 and self.max_tokens > 0
```

Encapsulation

Encapsulation bundles data and the methods that operate on it, hiding internal state from the outside world. In AI pipelines, this prevents external code from corrupting model state.

- Public attributes
- Protected (_attr)
- Private (__attr)
- Properties (@property)

Encapsulating model weights

```
class NeuralModel:
    def __init__(self):
        self.__weights = None        # private — never touched directly
        self._is_trained = False     # protected — for subclasses

    @property
    def is_trained(self) -> bool:
        return self._is_trained

    def load_weights(self, path: str) -> None:
        self.__weights = self._read_checkpoint(path)
        self._is_trained = True
```

SOLID Principles Applied to AI

SOLID is the gold standard for object-oriented design. Each principle addresses a specific failure mode that AI projects commonly hit as they scale.

S — Single Responsibility Principle (SRP)

Every class should have one reason to change. An AI class that loads data, normalizes it, trains a model, and logs metrics violates SRP catastrophically.

- DataLoader class
- Preprocessor class
- Trainer class
- Evaluator class

O — Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification. Add new model types by subclassing, not by editing the base training loop.

```
class BaseModel(ABC):
    @abstractmethod
    def predict(self, inputs: list) -> list: ...

class TransformerModel(BaseModel):        # extend, don't modify
    def predict(self, inputs: list) -> list:
        return self._run_inference(inputs)
```

L — Liskov Substitution Principle (LSP)

Subclasses must be substitutable for their base class. If your pipeline expects a BaseModel, any concrete subclass must work without special-casing.

I — Interface Segregation Principle (ISP)

Don't force classes to implement methods they don't need. Split large abstract interfaces into smaller, focused ones.

- Trainable (abstract)
- Predictable (abstract)
- Serializable (abstract)

D — Dependency Inversion Principle (DIP)

High-level modules (Trainer) should not depend on low-level modules (a specific optimizer). Both should depend on abstractions (BaseOptimizer interface). This makes swapping components trivial.

Inheritance & Polymorphism in AI

Inheritance — Building Model Hierarchies

Inheritance lets concrete model classes share common infrastructure (loading configs, logging, checkpointing) while specializing in their own architecture.

A practical ML model hierarchy

```
class BaseMLModel(ABC):
    def __init__(self, config: ModelConfig):
        self.config = config
        self.logger = logging.getLogger(self.__class__.__name__)

    @abstractmethod
    def fit(self, X, y): ...

    @abstractmethod
    def predict(self, X): ...

    def save(self, path: str):     # shared by ALL subclasses
        joblib.dump(self, path)
```

Polymorphism — One Interface, Many Behaviors

Polymorphism lets you write code that works with any object that satisfies a contract, regardless of its concrete type. This is the backbone of pluggable AI pipelines.

Polymorphic prediction — swap models without changing pipeline code

```
def run_pipeline(model: BaseMLModel, dataset: Dataset) -> Report:
    model.fit(dataset.X_train, dataset.y_train)
    preds = model.predict(dataset.X_test)   # works for ANY subclass
    return Evaluator.score(preds, dataset.y_test)

# Works identically for all three — no if/elif needed
run_pipeline(LogisticRegressionModel(cfg), ds)
run_pipeline(RandomForestModel(cfg), ds)
run_pipeline(TransformerModel(cfg), ds)
```

Abstract Base Classes (ABC) vs. Protocols

Mechanism	Use When	Example
ABC +	You control the class hierarchy; strict	BaseModel, BaseDataset

Mechanism	Use When	Example
@abstractmethod	contracts	
Protocol (structural typing)	You don't own the classes; duck typing preferred	Callable, Iterable, Sized
Dataclass	Primarily data containers with little behavior	ModelConfig, BatchResult
NamedTuple	Immutable records; config snapshots	TrainingStep, Checkpoint

Modular Code Architecture for AI Projects

Why Modularity Matters

A modular codebase is one where components can be developed, tested, replaced, and scaled independently. In AI, this means you can swap a model, upgrade a data source, or change evaluation logic without touching anything else.

Recommended Project Structure

```
ai_project/
├── data/
│   ├── __init__.py
│   ├── loader.py          # DataLoader, StreamingLoader
│   ├── preprocessor.py    # Normalizer, Tokenizer, Augmentor
│   └── schema.py          # Pydantic models for data contracts
├── models/
│   ├── __init__.py
│   ├── base.py            # BaseMLModel ABC
│   ├── classifiers.py     # LogReg, RandomForest, SVM
│   └── transformers.py    # BERTModel, GPTWrapper
├── training/
│   ├── trainer.py         # Trainer class
│   ├── callbacks.py       # EarlyStopping, ModelCheckpoint
│   └── scheduler.py       # LR schedulers
├── evaluation/
│   └── evaluator.py       # Metrics, confusion matrix, reports
├── serving/
│   └── predictor.py       # REST-ready inference wrapper
└── exceptions/
    └── ai_errors.py       # All custom exceptions
```

Dependency Injection in AI Pipelines

Instead of hardcoding dependencies inside a class (e.g., creating a specific model inside Trainer), inject them from outside. This makes every component independently testable.

```
class Trainer:
    def __init__(self,
                 model: BaseMLModel,
                 optimizer: BaseOptimizer,
                 loader:    BaseDataLoader):
        self.model     = model
        self.optimizer = optimizer
        self.loader    = loader

# Compose at the top level — swap any piece freely
trainer = Trainer(
```

```
    model     = TransformerModel(config),
    optimizer = AdamOptimizer(lr=1e-4),
    loader    = StreamingLoader(path),
)
```

Design Rule

Each module should have a single, clear public interface. If a module's __init__.py exports more than 5–7 symbols, it is doing too much and should be split further.

Design Patterns for AI Systems

Design patterns are reusable solutions to recurring architectural problems. Four patterns appear constantly in production AI systems.

1. Factory Pattern — Dynamic Model Selection

When the type of model to instantiate is determined at runtime (e.g., from a config file or CLI argument), use a Factory to centralize that logic rather than scattering if/elif chains everywhere.

```
class ModelFactory:
    _registry: dict[str, type] = {}

    @classmethod
    def register(cls, name: str, model_cls: type) -> None:
        cls._registry[name] = model_cls

    @classmethod
    def create(cls, name: str, config: ModelConfig) -> BaseMLModel:
        if name not in cls._registry:
            raise ModelNotFoundError(f'Unknown model: {name}')
        return cls._registry[name](config)
```

2. Strategy Pattern — Interchangeable Algorithms

The Strategy pattern lets you define a family of algorithms, encapsulate each one, and make them interchangeable. Classic AI use cases: loss functions, sampling strategies, tokenization methods.

- Define a BaseStrategy ABC with a single execute() method
- Each concrete strategy (e.g., GreedyDecoding, BeamSearch, NucleausSampling) implements execute()
- The LLM client holds a reference to a strategy, not a concrete implementation

3. Observer Pattern — Training Callbacks

Used extensively in deep learning frameworks (Keras callbacks, PyTorch Lightning hooks). The Trainer is the subject; callbacks (EarlyStopping, WandB logger, ModelCheckpoint) are observers.

```
class Trainer:
    def __init__(self, callbacks: list[BaseCallback] = None):
        self._callbacks = callbacks or []

    def _emit(self, event: str, **kwargs) -> None:
        for cb in self._callbacks:
            getattr(cb, event, lambda **k: None)(**kwargs)

    def fit(self, loader):
```

```
        self._emit('on_train_begin')
        for epoch, batch in enumerate(loader):
            loss = self._step(batch)
            self._emit('on_epoch_end', epoch=epoch, loss=loss)
```

4. Pipeline Pattern — Sequential Transformations

A Pipeline chains a sequence of steps where each step's output is the next step's input. Used in scikit-learn, Hugging Face, and almost every production data preprocessing workflow.

- Each step implements a transform(data) → data interface
- The Pipeline composes steps and handles errors between them
- Steps can be mixed: CPU preprocessing → GPU model → postprocessing

Error Handling Strategies

Why Error Handling is Critical in AI

AI systems operate on noisy, unpredictable real-world data. A missing feature column, an API timeout, a corrupt model checkpoint, or a distribution shift can all cause failures. Robust error handling is what separates production-grade AI from research prototypes.

Python's Exception Hierarchy

- BaseException
- Exception
- ValueError
- TypeError
- RuntimeError
- IOError / OSError

The try / except / else / finally Pattern

```
try:
    predictions = model.predict(batch)      # risky operation
except ModelNotLoadedError as e:
    logger.error('Model not ready: %s', e)   # specific, typed catch
    raise InferenceError('Cannot predict') from e
except (ValueError, TypeError) as e:
    logger.warning('Bad input shape: %s', e) # handle data issues
    predictions = fallback_model.predict(batch)
else:
    metrics.record(len(predictions))         # only runs if no exception
finally:
    batch.release()                          # always runs — cleanup
```

Error Handling Levels in an AI System

Layer	What to Handle	Strategy
Data Ingestion	Missing files, corrupt formats, schema violations	Validate early, fail fast with clear messages
Preprocessing	NaN values, out-of-range features, type mismatches	Catch, log, and either impute or skip the sample
Model Inference	OOM errors, shape mismatches, device errors	Retry with smaller batch; fall back to CPU
Output Validation	NaN outputs, out-of-distribution	Assert or raise; never serve

Layer	What to Handle	Strategy
	predictions	garbage silently
Infrastructure	Network timeouts, DB failures, API rate limits	Exponential backoff with circuit breaker

Advanced Error Handling Techniques

Graceful Degradation

A production AI system should degrade gracefully rather than crash. When the primary model fails, fall back to a simpler model, a cached result, or a rule-based default — and always alert the monitoring system.

```
class RobustPredictor:
    def predict(self, inputs: list) -> list:
        try:
            return self._primary_model.predict(inputs)
        except InferenceError as e:
            self._alerts.send(f'Primary model failed: {e}')
            return self._fallback_model.predict(inputs)
```

Context Managers for Resource Safety

Use Python's context manager protocol to guarantee resource cleanup (GPU memory, file handles, DB connections) even when exceptions occur mid-operation.

```
class ModelSession:
    def __enter__(self):
        self._session = self._load_model()
        return self._session

    def __exit__(self, exc_type, exc_val, exc_tb):
        self._session.close()   # runs even if exception was raised
        return False            # re-raise any exception

with ModelSession(config) as session:
    result = session.infer(data)
```

Retry Patterns with Exponential Backoff

Network calls to model APIs (OpenAI, Vertex AI, SageMaker) fail transiently. A retry decorator with exponential backoff is standard in AI serving code.

```
def retry(max_attempts: int = 3, base_delay: float = 1.0):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except TransientAPIError as e:
                    if attempt == max_attempts - 1: raise
                    time.sleep(base_delay * (2 ** attempt))
        return wrapper
    return decorator
```

Logging Best Practices in AI Systems

- Use structured logging (JSON) in production — enables Elasticsearch/Splunk queries
- Log at the right level: DEBUG for tensor shapes, INFO for pipeline stages, ERROR for failures
- Always include context: batch ID, model version, input shape, elapsed time
- Never log raw PII (user prompts, personal data) — mask or hash before logging
- Use correlation IDs to trace a single request across multiple services

Custom Exception Hierarchies

Why Custom Exceptions?

Catching generic exceptions (except Exception) is an antipattern. It swallows bugs and makes debugging a nightmare. Custom exceptions carry semantic meaning — they tell the caller exactly what went wrong and what they can do about it.

> **Core Principle**
> Every distinct failure mode in your AI system deserves its own exception type. If you have to read the error message to understand what category of problem occurred, your exception hierarchy is too shallow.

Designing a Full AI Exception Hierarchy

```
# Base — all AI errors inherit from this
class AIApplicationError(Exception):
    def __init__(self, message: str, context: dict = None):
        super().__init__(message)
        self.context = context or {}

# ── Data Errors ──────────────────────────
class DataError(AIApplicationError): ...
class SchemaValidationError(DataError): ...
class MissingFeatureError(DataError): ...
class DataCorruptionError(DataError): ...

# ── Model Errors ─────────────────────────
class ModelError(AIApplicationError): ...
class ModelNotLoadedError(ModelError): ...
class ModelNotTrainedError(ModelError): ...
class ModelNotFoundError(ModelError): ...

# ── Inference Errors ─────────────────────
class InferenceError(AIApplicationError): ...
class ShapeMismatchError(InferenceError): ...
class DeviceError(InferenceError): ...

# ── Infrastructure Errors ────────────────
class InfrastructureError(AIApplicationError): ...
class TransientAPIError(InfrastructureError): ...
class ConfigurationError(InfrastructureError): ...
```

Exception Chaining, Context & Best Practices

Exception Chaining with 'raise ... from'

When a low-level exception (e.g., a FileNotFoundError) causes a high-level failure (e.g., a ModelNotLoadedError), chain them. This preserves the full traceback so engineers can diagnose both the cause and the consequence.

```
def load_model(path: str) -> BaseMLModel:
    try:
        with open(path, 'rb') as f:
            return pickle.load(f)
    except FileNotFoundError as e:
        raise ModelNotLoadedError(
            f'Checkpoint not found at {path}',
            context={'path': path, 'reason': 'file_missing'}
        ) from e   # <-- chains the original cause
```

Enriching Exceptions with Context

Custom exceptions become far more useful when they carry structured context — machine-readable metadata that monitoring tools, log aggregators, and on-call engineers can act on immediately.

```
class InferenceError(AIApplicationError):
    def __init__(self, message, batch_id=None,
                 input_shape=None, model_version=None):
        super().__init__(message, context={
            'batch_id':      batch_id,
            'input_shape':   str(input_shape),
            'model_version': model_version,
        })
        self.batch_id      = batch_id
        self.input_shape   = input_shape
        self.model_version = model_version
```

Exception Handling Anti-Patterns to Avoid

Anti-Pattern	Why It's Harmful	Correct Approach
except Exception: pass	Silences all errors including bugs	Catch specific types; always log
raise Exception('msg')	No semantic type; callers can't handle selectively	Raise a typed custom exception
Catch-and-print without re-raise	Execution continues in an unknown state	Log, then either recover or re-raise
Catching in a loop without break	Infinite retry loops that hammer APIs	Use backoff + max_attempts

Anti-Pattern	Why It's Harmful	Correct Approach
Storing exceptions as return values	Forces every caller to check for error type	Let exceptions propagate; use Result type only if explicit

Production Rule

In serving code, never let an unhandled exception propagate to the user. Every public API endpoint should have a top-level exception handler that converts any AIApplicationError into a structured error response with a request ID.

Testing OOP AI Code

Unit Testing Strategies

Good OOP design makes unit testing straightforward. Each class has a single responsibility, dependencies are injected (and therefore mockable), and public interfaces are clearly defined.

- Test one class at a time
- Use pytest fixtures
- Test exception paths explicitly
- Property-based testing

Testing that the correct exception is raised

```
import pytest

def test_predict_raises_when_not_trained():
    model = TransformerModel(ModelConfig('bert', 512, 0.7))
    with pytest.raises(ModelNotTrainedError) as exc_info:
        model.predict(['hello world'])
    assert 'not trained' in str(exc_info.value)
```

Key Takeaways

OOP Foundations

Use classes to encapsulate AI concepts. Apply SOLID principles to keep each class focused, extensible, and independently testable.

Modular Architecture

Organize code into cohesive modules with clear public interfaces. Use dependency injection so components can be swapped and tested in isolation.

Error Handling

Use typed exceptions, chain causes, and carry context. Design for graceful degradation — your serving code must never crash silently.

Custom Exception Hierarchies

Build a domain-specific exception tree rooted in a single AIApplicationError base. Catch the most specific type you can; always log context; never swallow errors.

— End of Presentation —